

SE3112 – Advanced Software Engineering

Lecture 01 - Introduction

Jeewaka Perera

Updated July 2026

Adapted From the Text book Slides by Paul Ammann & Jeff Offutt (Chapter 1-

5)

Module Introduction

□ Lecture Delivery

- Lecture 02 Hours
- Practical 02 Hour

□ Lecture Panel

- Lecture - Mr. Jeewaka Perera

□ Practical Delivery Panel

- Mr. Dinal Senadheera
- Ms. Nivinya Abeysinghe

□ Evaluation Criteria

Evaluation	Marks
In Class Quizzes	10%
Mid Term Test	20%
Software Testing Project	10%
Final Examination	60%

Tentative Delivery Plan

- Week 1 - Introduction to Software Testing
- Week 2 - Graph Coverage
- Week 3 - Logic Coverage
- Week 4 - Input Space Partitioning
- Week 5 - Syntax
- Week 6 - Test Doubles/Mock Testing
- Week 7 - Non-Functional Testing and Tools
- Week 9 - Introduction to TDD
- Week 10 - Introduction to BDD
- Week 11 - Software Metrics
- Week 12 - Software Reliability and Fault Tolerance
- Week 13 - Productivity and Cost Estimation

Testing in the 21st Century

□ Software defines **behavior**

- network routers, finance, switching networks, other infrastructure

□ Today's software **market** :

- is much **bigger**
- is more **competitive**
- has more **users**

Industry is going through a revolution in what testing means to the success of software products

□ **Embedded Control Applications**

- airplanes, air traffic control
- spaceships
- watches
- ovens
- remote controllers
- PDAs
- memory seats
- DVD players
- garage door openers
- cell phones

□ **Agile** processes put increased pressure on testers

- **Programmers** must **unit** test – with no training or education!
- Tests are key to **functional requirements** – but who builds those



Quote due to Dr. Mark Harman

Software Faults, Errors & Failures

- **Software Fault** : A static defect in the software
- **Software Failure** : External, incorrect behavior with respect to the requirements or other description of the expected behavior
- **Software Error** : An incorrect internal state that is the manifestation of some fault

Faults in software are equivalent to design mistakes in hardware.

Software does not degrade.

Fault and Failure Example

- A patient gives a doctor a list of **symptoms**
 - Failures
- The doctor tries to diagnose the root cause, the **ailment**
 - Fault
- The doctor may look for **anomalous internal conditions** (high blood pressure, irregular heartbeat, bacteria in the blood stream)
 - Errors

Most medical problems result from external attacks (bacteria, viruses) or physical degradation as we age. Software faults were there at the beginning and do not “appear” when a part wears out.

A Concrete Example

Fault: Should start searching at 0, not 1

```
public static int numZero (int [ ] arr)
{ // Effects: If arr is null throw NullPointerException
  // else return the number of occurrences of 0 in arr
  int count = 0;
  for (int i = 1; i < arr.length; i++)
  {
    if (arr [ i ] == 0)
    {
      count++;
    }
  }
  return count;
}
```

Test 1
[2, 7, 0]
Expected: 1
Actual: 1

Error: i is 1, not 0, on the first iteration
Failure: none

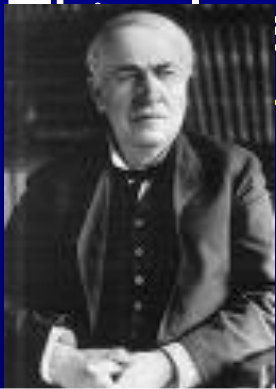
Test 2
[0, 2, 7]
Expected: 1
Actual: 0

Error: i is 1, not 0
Error propagates to the variable count
Failure: count is 0 at the return statement

The Term Bug

- **Bug** is used informally
- Sometimes **speakers mean fault**, sometimes **error**, sometimes **failure** ... often the speaker doesn't know what it means !

- **Speakers will try to use v**
ambiguous BUG **meaning**



“It has been just so in all of my inventions. The first step is an intuition, and comes with a burst, then difficulties arise—this thing gives out and *[it is]* then that '**Bugs**'—as such little faults and difficulties are called—show themselves and months of intense watching, study and labor are requisite. . .” – Thomas Edison



- **Speakers have precise, defined,**

“an analyzing process must equally have been performed in order to furnish the Analytical Engine with the necessary operative data; and that herein may also lie a possible source of **error**. Granted that the actual mechanism is unerring in its processes, the cards may give it wrong orders.” – Ada, Countess Lovelace (notes on Babbage's Analytical Engine)

Spectacular Software Failures

- ❑ **NASA's Mars lander: September 1999, crashed due to a units integration fault**

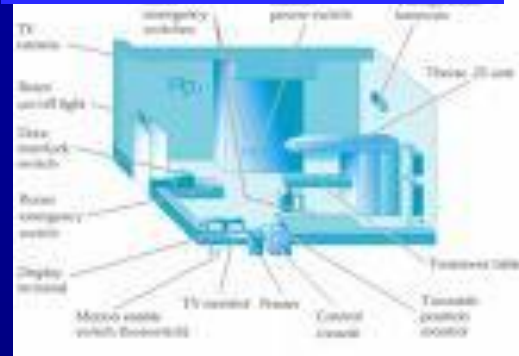


Mars Polar
Lander crash
site?

- ❑ **THERAC-25 radiation machine : Poor testing of safety-critical software can cost lives : 3 patients were killed**
- ❑ **Ariane 5 explosion : Millions of \$\$**
- ❑ **Intel's Pentium FDIV fault : Public relations nightmare**



THERAC-25 design



Ariane 5:
exception-handling
bug : forced self
destruct on maiden
flight (64-bit to 16-bit
conversion: about
370 million \$ lost)

**We need our software to be
dependable**

Testing is one way to assess

Northeast Blackout of 2003

508 generating
units and 256
power plants
shut down

Affected 10
million people in
Ontario, Canada

Affected 40
million people in
8 US states

Financial losses
of
\$6 Billion USD

The **alarm system** in the energy management system
failed due to a software error and operators were not
informed of the power overload in the system



Costly Software Failures

- **NIST report, “The Economic Impacts of Inadequate Infrastructure for Software Testing” (2002)**
 - Inadequate software testing costs the US alone between \$22 and \$59 billion annually
 - Better approaches could cut this amount in half
- **Huge losses due to web application failures**
 - Financial services : \$6.5 million per hour (just in USA!)
 - Credit card sales applications : \$2.4 million per hour (in USA)

World-wide monetary loss due to poor software is staggering

into a double discount

- **2007 : Symantec says that most security vulnerabilities are due to faulty software**

Spectacular software Failures

- **Boeing A220** : Engines failed after software update allowed excessive vibrations
- **Boeing 737 Max** : Crashed due to overly aggressive software flight overrides (MCAS)
- **Toyota brakes** : Dozens dead, thousands of



- **Healthcare website** : Crashed repeatedly on launch—never load tested

- **Northeast blackout** : 50 million people
\$6 billion USD lost ... alarm system fa



**Software testers try to find faults
before the faults find users**

Testing in the 21st Century

- More **safety** critical, **real-time** software
- **Embedded** software is ubiquitous ... check your pockets
- **Enterprise** applications means bigger programs, more users
- Paradoxically, free software **increases** our expectations !
- **Security** is now all about software faults
 - **Secure** software is **reliable** software
- The **web** offers a new deployment platform
 - Very **competitive** and very **available** to more users
 - Web apps are distributed
 - **Web apps** must be highly reliable

Industry desperately needs our inventions !

The True Cost of Software Failure

**Fail watch analyzed news articles for
2016**

- **606 reported software failures**
- **Impacted half the world's population**
- **Cost a combined \$1.7 trillion US dollars**

**Poor software is a significant
drag on the world's economy**

**Not to mention
frustrating**

What Does This Mean?

Software testing is getting more important

**What are we trying to do when we test ?
What are our goals ?**

Validation & Verification (*IEEE*)

- **Validation** : The process of evaluating software at the end of software development to ensure compliance with intended usage
- **Verification** : The process of determining whether the products of a given phase of the software development process fulfill the requirements established during the previous phase

IV&V stands for “*independent verification and validation*”

Testing Goals Based on Test Process Maturity

- **Level 0** : There's no difference between testing and debugging
- **Level 1** :The purpose of testing is to show correctness
- **Level 2** :The purpose of testing is to show that the software doesn't work
- **Level 3** :The purpose of testing is not to prove anything specific, but to reduce the risk of using the software
- **Level 4** :Testing is a mental discipline that helps all IT professionals develop higher quality software

Level 0 Thinking

- Testing is the **same** as debugging
- Does not distinguish between incorrect **behavior** and mistakes in the program
- Does not help develop software that is **reliable** or **safe**

This is what we teach undergraduate CS majors

Level 1 Thinking

- Purpose is to show correctness
- Correctness is impossible to achieve
- What do we know if no failures?
 - Good software or bad tests?
- Test engineers have no:
 - Strict goal
 - Real stopping rule
 - Formal test technique
 - Test managers are powerless

This is what hardware engineers often expect

Level 2 Thinking

- Purpose is to show failures
- Looking for failures is a negative activity
- Puts testers and developers into an adversarial relationship
- What if there are no failures?

This describes most software companies.

How can we move to a team approach ??

Level 3 Thinking

- Testing can only show the **presence of failures**
- Whenever we use software, we incur some **risk**
- Risk may be **small** and consequences unimportant
- Risk may be **great** and consequences catastrophic
- Testers and developers cooperate to **reduce risk**

This describes a few “enlightened” software companies

Level 4 Thinking

A mental discipline that increases quality

- Testing is only **one way** to increase quality
- Test engineers can become **technical leaders** of the project
- Primary responsibility to **measure and improve** software quality
- Their expertise should **help the developers**

This is the way “traditional” engineering works

Where Are You?

Are you at level 0, 1, or 2 ?

**Is your organization at work at level
0, 1, or 2 ?**

Or 3?

**We hope to teach you to become
“change agents” in your workplace ...**

Advocates for level 4 thinking

Tactical Goals : Why Each Test ?

If you don't know why you're conducting each test, it won't be very helpful

- Written test objectives and requirements must be documented
- What are your planned coverage levels?
- How much testing is enough?
- Common objective – spend the budget ... test until the ship-date ...
 - Sometimes called the “date criterion”

Here! Test This!

Offutt's first “professional” job



A stack of computer printouts—and no documentation

Why Each Test ?

If you don't start planning for each test when the functional requirements are formed, you'll never know why you're conducting the test

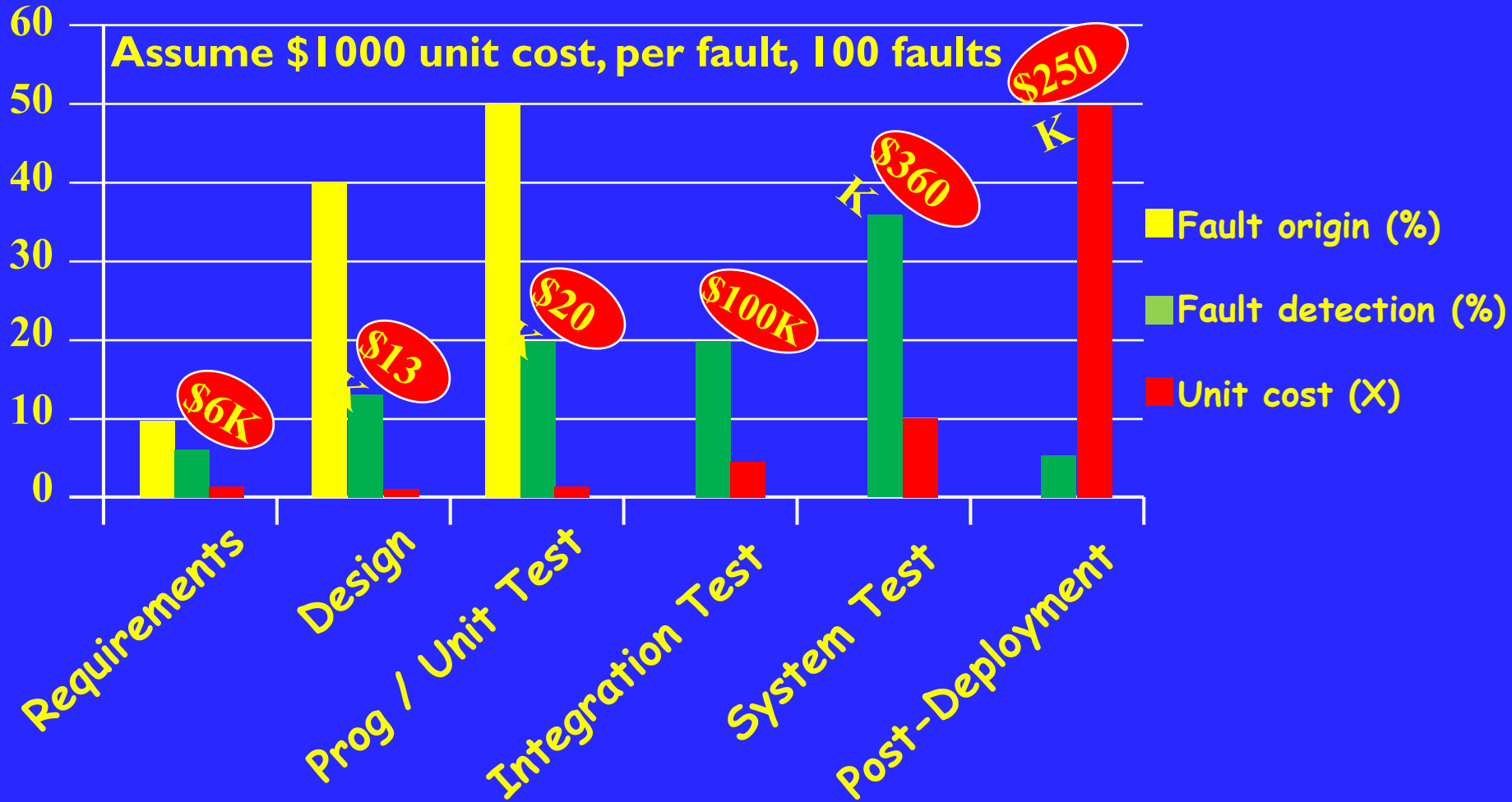
- 1980: “The software shall be easily **maintainable**”
- Threshold **reliability** requirements?
- What fact does each test try to **verify**?
- **Requirements** definition teams need testers!

Cost of Not Testing

Poor Program Managers might say: "Testing is too expensive."

- Testing is the **most time consuming** and expensive part of software development
- Not testing is even **more expensive**
- If we have too little testing effort early, the cost of testing **increases**
- Planning for testing after development is **prohibitively** expensive

Cost of Late Testing



Software Engineering Institute; Carnegie Mellon University; Handbook CMU/SEI-96-HB-002

Model-Driven Test Design

Complexity of Testing Software

- No other engineering field builds products as **complicated** as software
- The term **correctness** has no meaning
 - Is a **building** correct?
 - Is a **car** correct?
 - Is a **subway** system correct?
- Like other engineers, we must use **abstraction to manage complexity**
 - This is the purpose of the **model-driven test design** process
 - The “model” is an abstract structure

Software Testing Foundations

**Testing can only show the presence
of failures**

Not their absence

Testing & Debugging

- **Testing** : Evaluating software by observing its execution
- **Test Failure** : Execution of a test that results in a software failure
- **Debugging** : The process of finding a fault given a failure

Not all inputs will “trigger” a fault into causing a failure

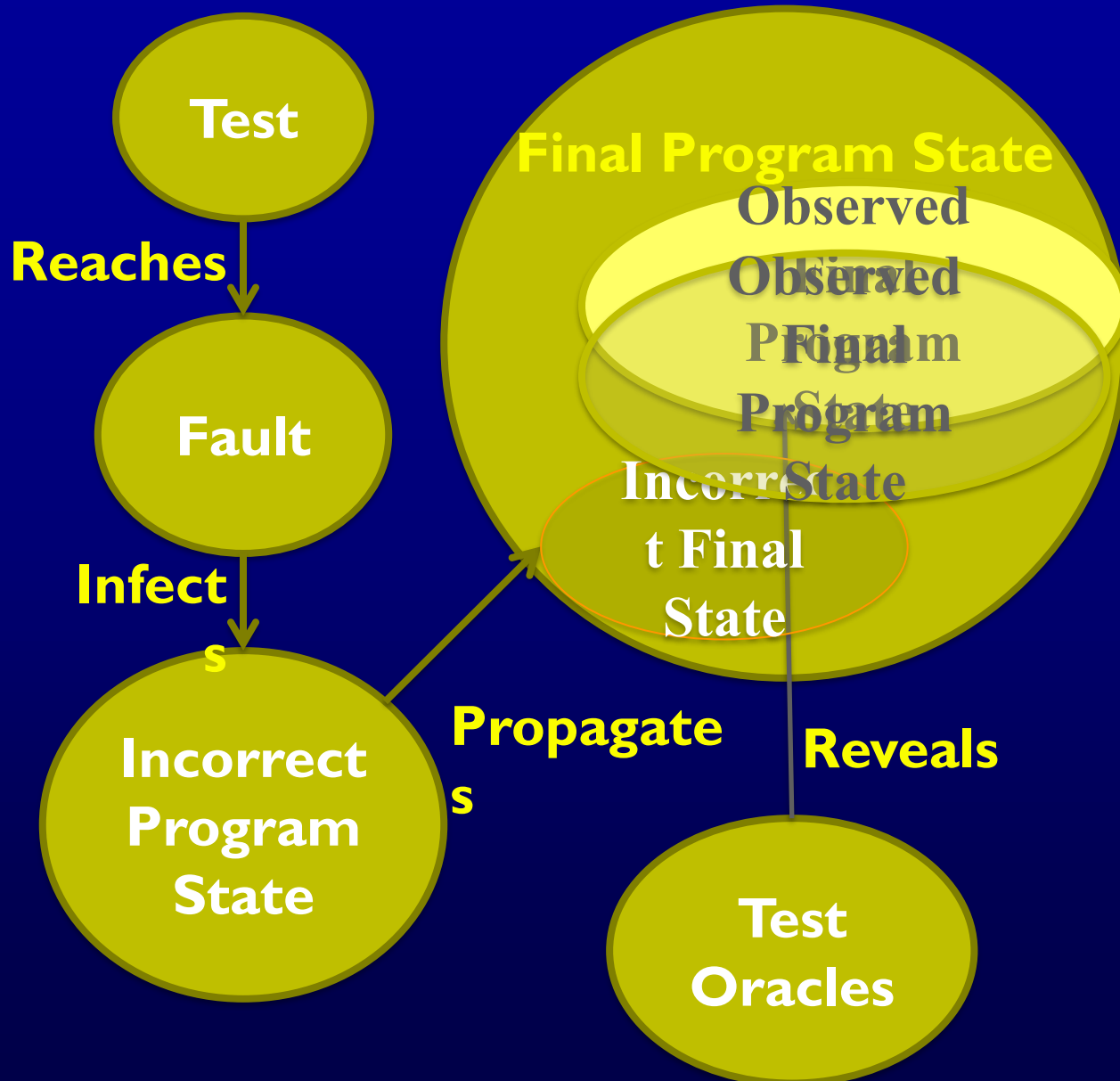
Fault & Failure Model (RIPR)

Four conditions necessary for a failure to be observed

1. **Reachability** : The location or locations in the program that contain the fault must be reached
2. **Infection** : The state of the program must be incorrect
3. **Propagation** : The infected state must cause some output or final state of the program to be incorrect
4. **Reveal** : The tester must observe part of the incorrect portion of the program state

RIPR Model

- **Reachability**
- **Infection**
- **Propagation**
- **Revealability**

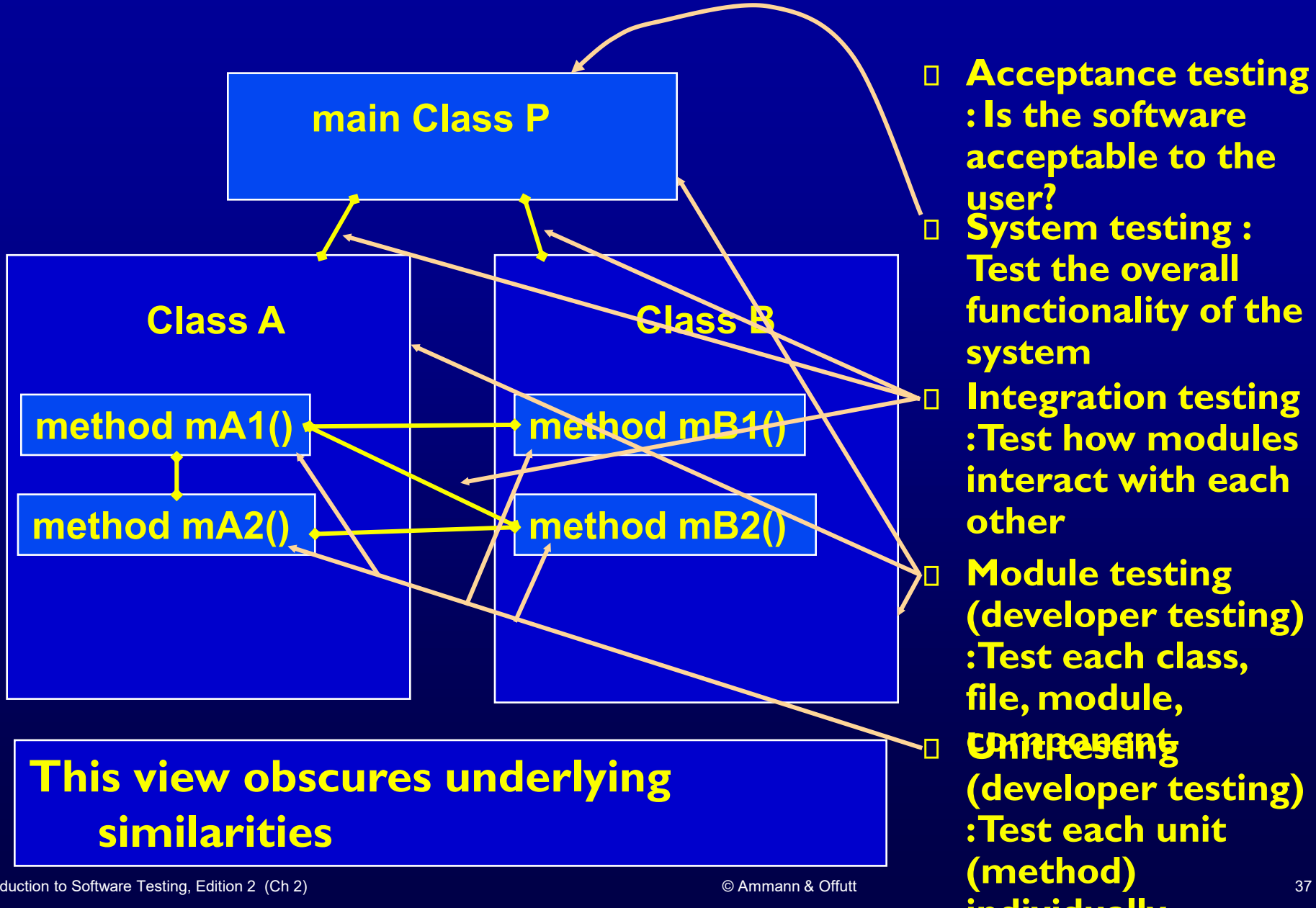


Software Testing Activitie

- Test Engineer : An IT professional who is in charge of one or more technical test activities
 - Designing test inputs
 - Producing test values
 - Running test scripts
 - Analyzing results
 - Reporting results to developers and managers

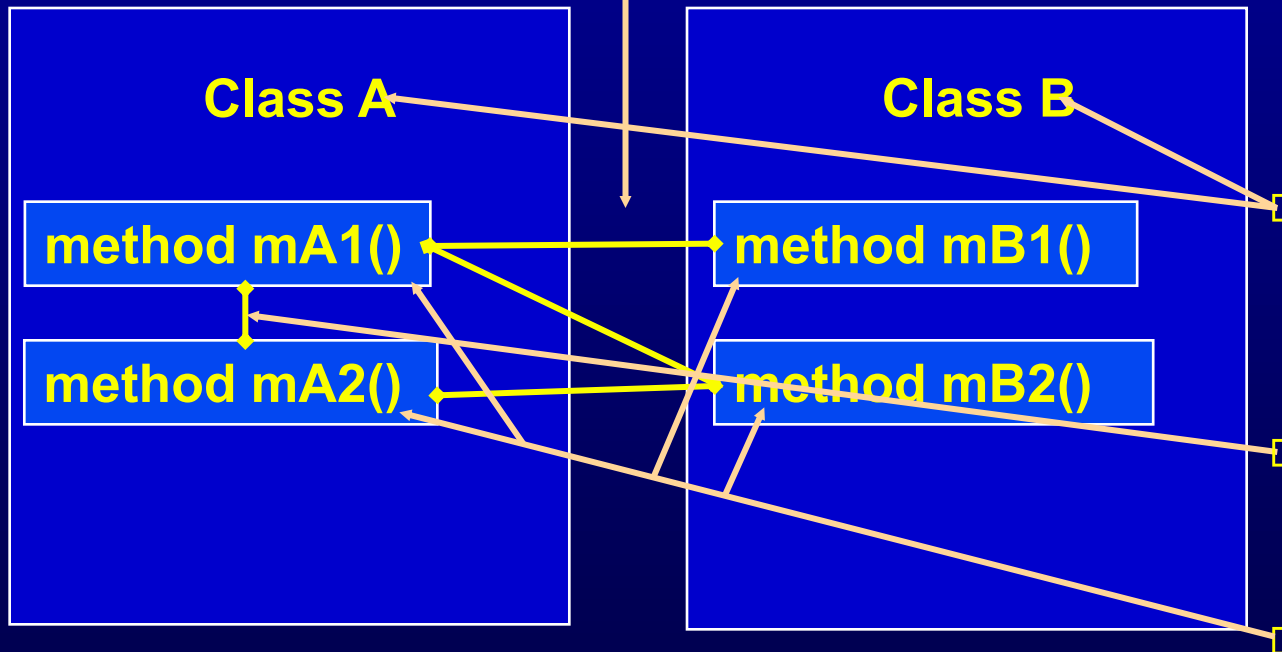
- Test Manager : In charge of one or more test engineers
 - Sets test policies and processes
 - Interacts with other managers on the project
 - Otherwise supports the engineers

Traditional Testing Levels (2.3)



Object-Oriented Testing Levels

- **Inter-class testing :**
Test multiple
classes together



- **Intra-class testing :**
Test an entire class
as sequences of
calls

- **Inter-method
testing :** Test pairs of
methods in the
same class

- **Intra-method
testing :** Test each
method individually

Coverage Criteria

- Even small programs have **too many inputs** to fully test them all
 - **private static double computeAverage (int A, int B, int C)**
 - On a 32-bit machine, each variable has over **4 billion** possible values
 - Over **80 octillion possible tests!!**
 - Input space might as well be infinite
- Testers **search** a huge input space
 - Trying to find the **fewest inputs** that will find the **most problems**
- **Coverage criteria** give structured, practical ways to search the input space
 - **Search** the input space thoroughly
 - **Not much overlap** in the tests

Advantages of Coverage Criteria

- Maximize the “bang for the buck”
- Provide **traceability** from software artifacts to tests
 - Source, requirements, design models, ...
- Make **regression testing** easier
- Gives testers a “**stopping rule**” ... when testing is finished
- Can be well supported with powerful **tools**

Test Requirements and Criteria

- **Test Criterion** : A collection of rules and a process that define test requirements
 - Cover every statement
 - Cover every functional requirement
- **Test Requirements** : Specific things that must be satisfied or covered during testing
 - Each statement might be a test requirement
 - Each functional requirement might be a test requirement

Testing researchers have defined dozens of criteria, but they are all really just a few criteria on four types of structures ...

1. Input domains
2. Graphs

3. Logic expressions
4. Syntax descriptions

Old View : Colored Boxes

- **Black-box testing** : Derive tests from external descriptions of the software, including specifications, requirements, and design
- **White-box testing** : Derive tests from the source code internals of the software, specifically including branches, individual conditions, and statements
- **Model-based testing** : Derive tests from a model of the software (such as a UML diagram)

MDTD makes these distinctions less important.

The more general question is:

from what abstraction level do we derive tests?

Model-Driven Test Design

- *Test Design* is the process of designing input values that will effectively test software
- Test design is one of **several activities** for testing software
 - Most **mathematical**
 - Most **technically** challenging

Types of Test Activities

- Testing can be broken up into **four** general types of activities

1. **Test Design** → **I.a) Criteria-based**
2. **Test Automation** → **I.b) Human-based**
3. **Test Execution**
4. **Test Evaluation**

- Each type of activity requires different **skills**, background **knowledge**, **education** and **training**
- No reasonable software development organization uses the same people for requirements, design, implementation, integration and configuration control

Why do test organizations still use the same people for all four test activities??

This clearly wastes resources

1. Test Design—(a) Criteria-Based

Design test values to satisfy coverage criteria or other engineering goal

- This is the **most technical** job in software testing
- Requires **knowledge** of :
 - Discrete math
 - Programming
 - Testing
- Requires much of a **traditional CS** degree
- This is **intellectually** stimulating, rewarding, and challenging
- Test design is analogous to **software architecture** on the development side
- Using people who are not qualified to design tests is a sure way to get **ineffective tests**

1. Test Design—(b) Human-Based

Design test values based on domain knowledge of the program and human knowledge of testing

- This is much **harder** than it may seem to developers
- Criteria-based approaches can be blind to special situations
- Requires **knowledge** of :
 - Domain, testing, and user interfaces
- Requires almost **no traditional CS**
 - A background in the **domain** of the software is essential
 - An **empirical background** is very helpful (biology, psychology, ...)
 - A **logic background** is very helpful (law, philosophy, math, ...)
- This is **intellectually** stimulating, rewarding, and challenging
 - But not to typical CS majors – they want to solve problems and build things

2. Test Automation

Embed test values into executable scripts

- This is slightly **less technical**
- Requires knowledge of **programming**
- Requires very **little theory**
- Often requires solutions to difficult problems related to **observability** and **controllability**
- Can be **boring** for test designers
- Programming is out of reach for many **domain experts**
- Who is responsible for determining and embedding the **expected outputs** ?
 - **Test designers** may not always know the expected outputs
 - **Test evaluators** need to get involved early to help with this

3. Test Execution

Run tests on the software and record the results

- This is **easy** – and trivial if the tests are well automated
- Requires basic **computer skills**
 - Interns
 - Employees with no technical background
- Asking qualified test **designers** to execute tests is a sure way to convince them to look for a **development job**
- If, for example, GUI tests are not well automated, this requires a lot of **manual labor**
- Test executors have to be very **careful** and **meticulous** with bookkeeping

4. Test Evaluation

Evaluate results of testing, report to developers

- This is much **harder** than it may seem
- Requires **knowledge** of :
 - Domain
 - Testing
 - User interfaces and psychology
- Usually requires almost **no traditional CS**
 - A background in the **domain** of the software is essential
 - An **empirical background** is very helpful (biology, psychology, ...)
 - A **logic background** is very helpful (law, philosophy, math, ...)
- This is **intellectually** stimulating, rewarding, and challenging
 - But not to typical CS majors – they want to solve problems and build things

Other Activities

- **Test management** : Sets policy, organizes team, interfaces with development, chooses criteria, decides how much automation is needed, ...
- **Test maintenance** : **Save tests for reuse** as software evolves
 - Requires cooperation of test **designers and automators**
 - Deciding when to trim the test suite is partly policy and partly technical – and in general, **very hard** !
 - Tests should be put in **configuration control**
- **Test documentation** : All parties participate
 - Each test must document “**why**” – criterion and test requirement satisfied or a rationale for human-designed tests
 - Ensure **traceability** throughout the process
 - Keep **documentation** in the automated tests

Organizing the Team

- A mature test organization needs **only one test designer** to work with several test automators, executors and evaluators
- **Improved automation** will reduce the number of test executors
 - Theoretically to zero ... but not in practice
- Putting the **wrong** people on the **wrong** tasks leads to **inefficiency**, low **job satisfaction** and low **job performance**
 - A qualified test designer will be **bored** with other tasks and look for a job in development
 - A qualified test evaluator will **not understand** the benefits of test
- Test **Many test teams use the same people for ALL FOUR activities !! must**

Applying Test Activities

**To use our people effectively
and to test efficiently
we need a process that**

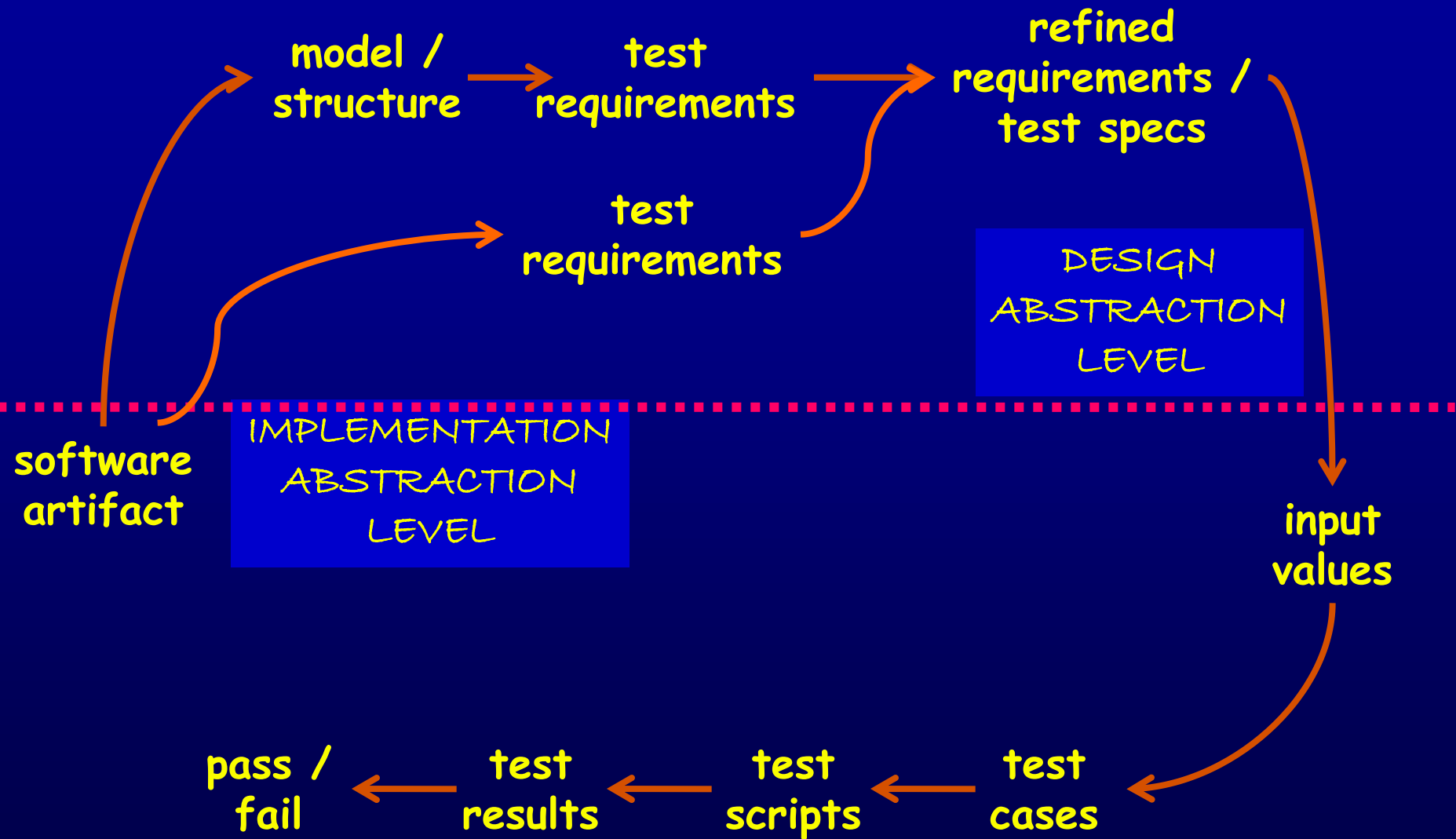
**lets test designers
raise their level of abstraction**

Using MDTD in Practice

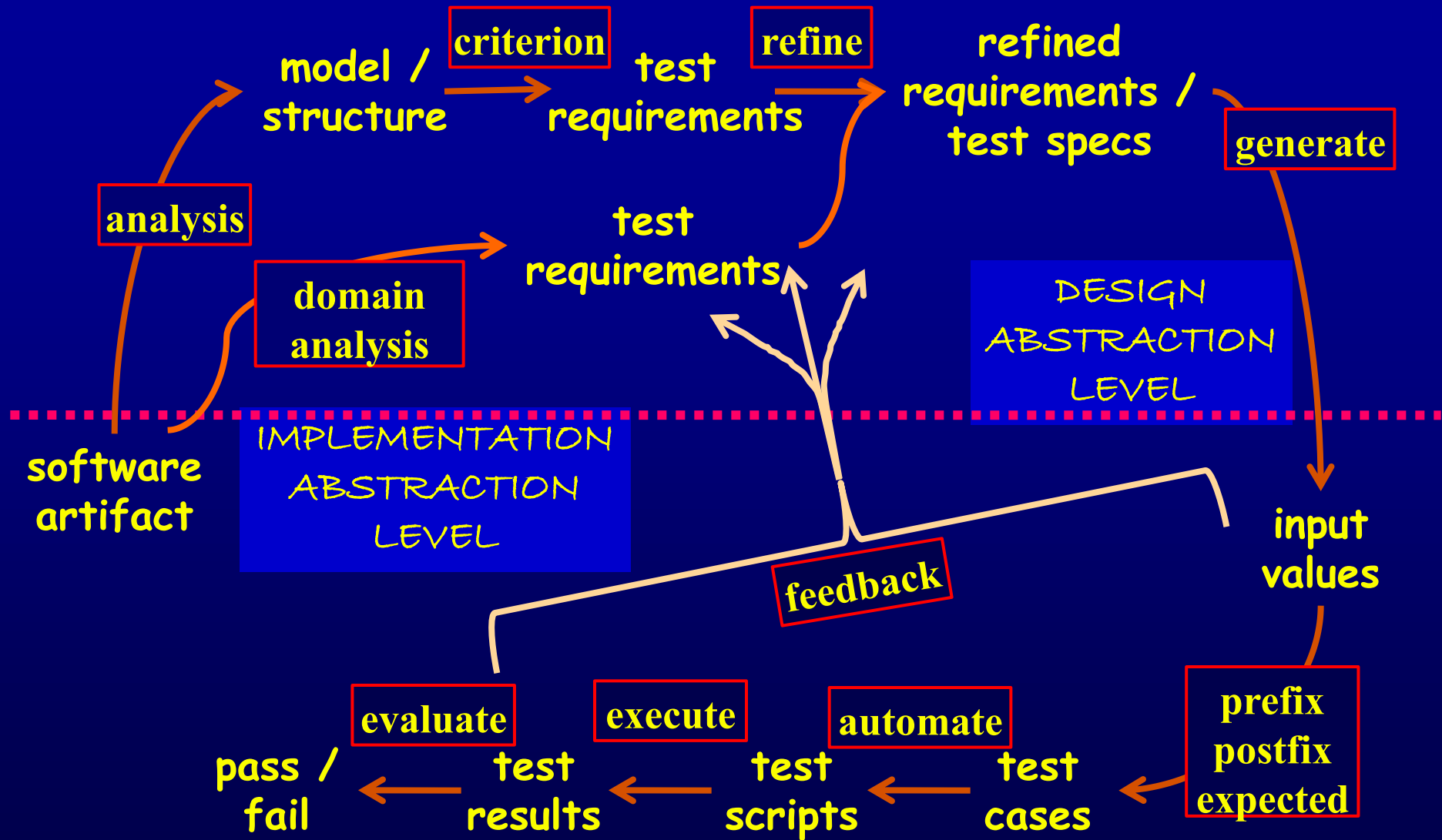
- This approach lets **one test designer** do the math
- Then traditional **testers** and **programmers** can do their parts
 - Find values
 - Automate the tests
 - Run the tests
 - Evaluate the tests
- Just like in **traditional engineering** ... an engineer constructs models with calculus, then gives direction to carpenters, electricians, technicians, ...

Test designers become technical experts

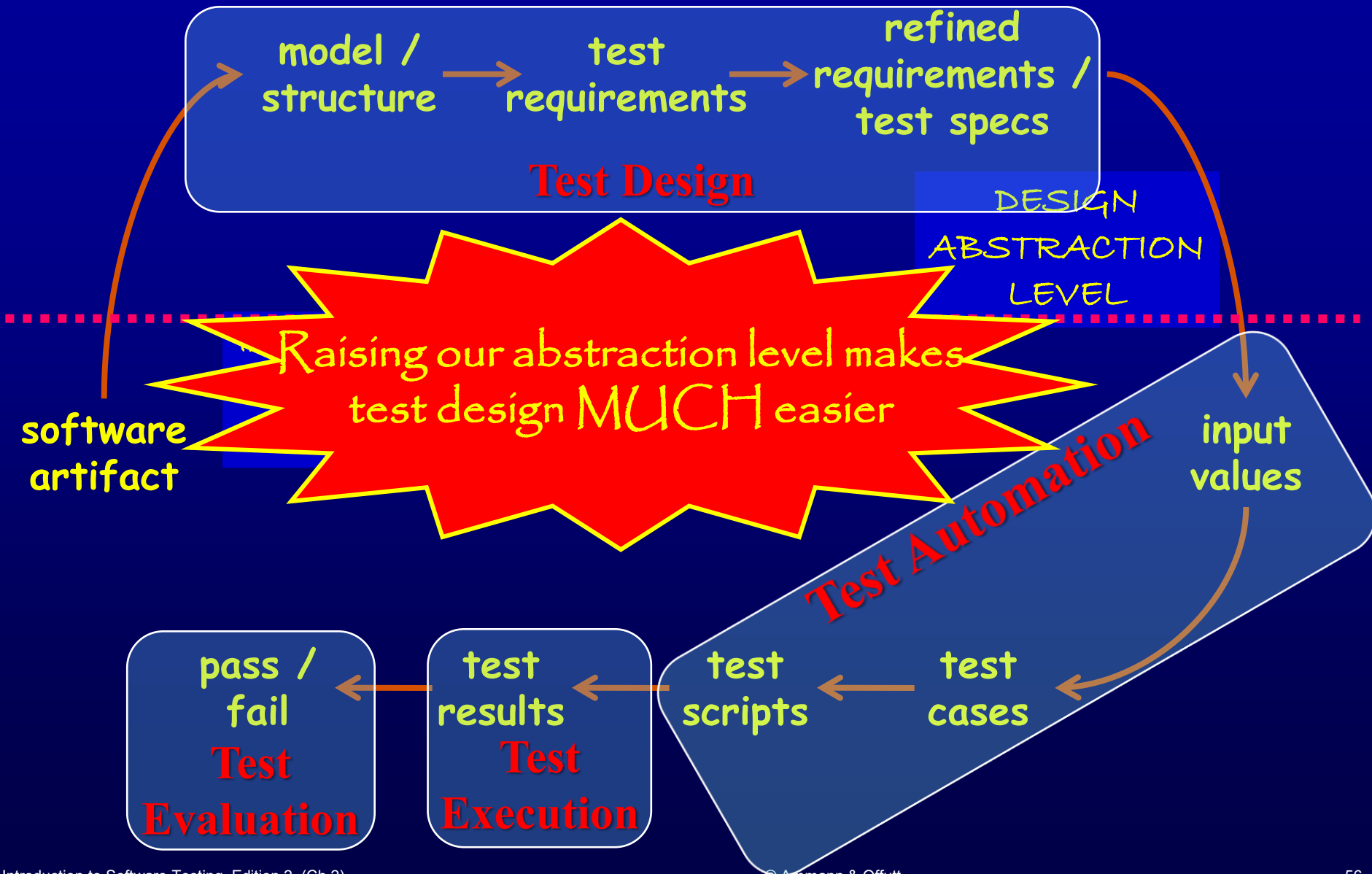
Model-Driven Test Design



Model-Driven Test Design – Steps



Model-Driven Test Design–Activities

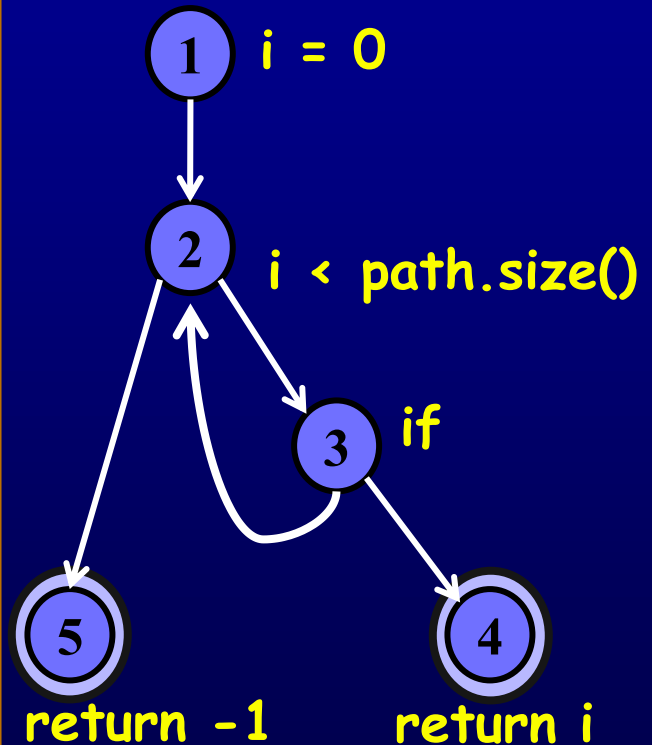


Small Illustrative Example

Software Artifact : Java Method

```
/**
 * Return index of node n at the
 * first position it appears,
 * -1 if it is not present
 */
public int indexOf (Node n)
{
    for (int i=0; i < path.size(); i++)
        if (path.get(i).equals(n))
            return i;
    return -1;
}
```

Control Flow Graph

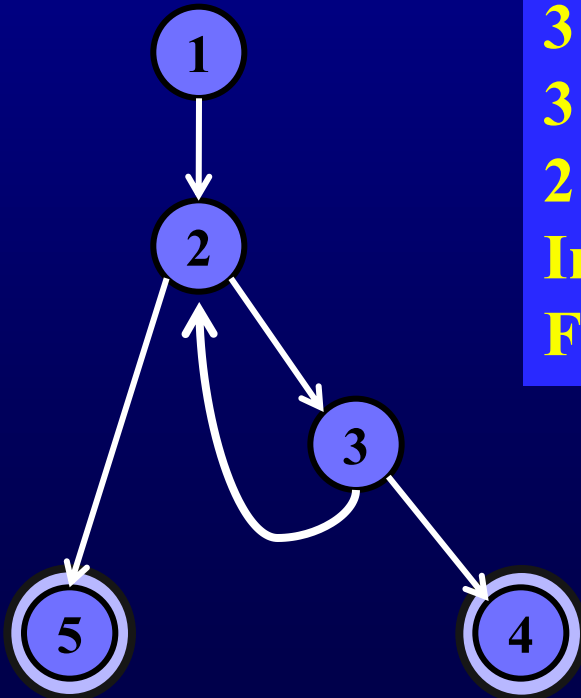


Example (2)

Support tool for graph coverage

<http://www.cs.gmu.edu/~offutt/softwaretest/>

**Graph
Abstract version**



Edges

1 2

2 3

3 2

3 4

2 5

Initial Node: 1

Final Nodes: 4, 5

**6 requirements for
Edge-Pair Coverage**

1. [1, 2, 3]

2. [1, 2, 5]

3. [2, 3, 4]

4. [2, 3, 2]

5. [3, 2, 3]

6. [3, 2, 5]

Test Paths

[1, 2, 5]

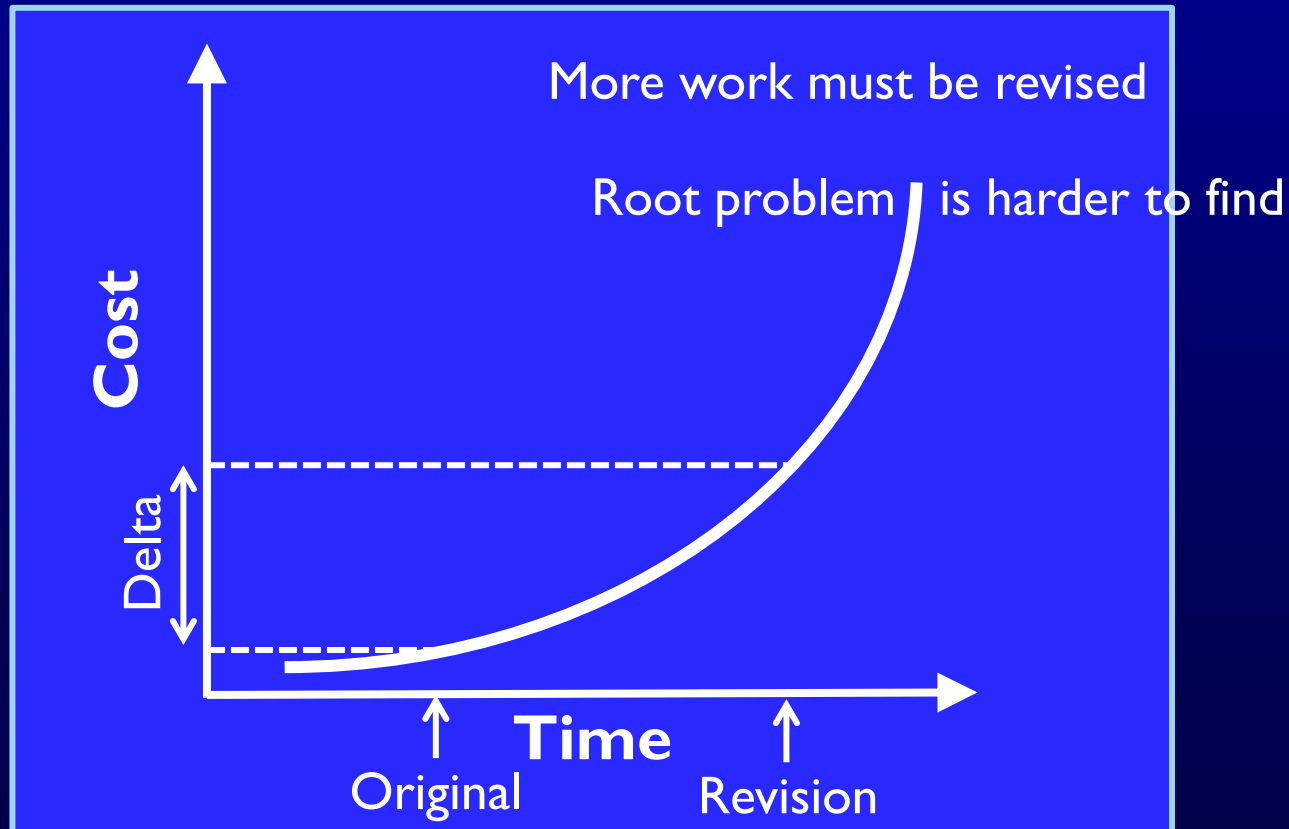
[1, 2, 3, 2, 5]

[1, 2, 3, 2, 3, 4]

Find values ...

The Increased Emphasis on Testing

- Philosophy of **traditional** software development methods
 - **Upfront** analysis
 - Extensive **modeling**
 - Reveal **problems** as early as possible



Traditional Assumptions

1. Modeling and analysis can identify potential problems early in development

2. Savings implied by the cost-of-change curve justify the cost of modeling and analysis over the life of the project

- These are true if requirements are always complete and current
- But those annoying customers keep changing their minds!
 - Humans are naturally good at approximating
 - But pretty bad at perfecting
- These two assumptions have made software engineering frustrating and difficult for decades

Thus, agile methods ...

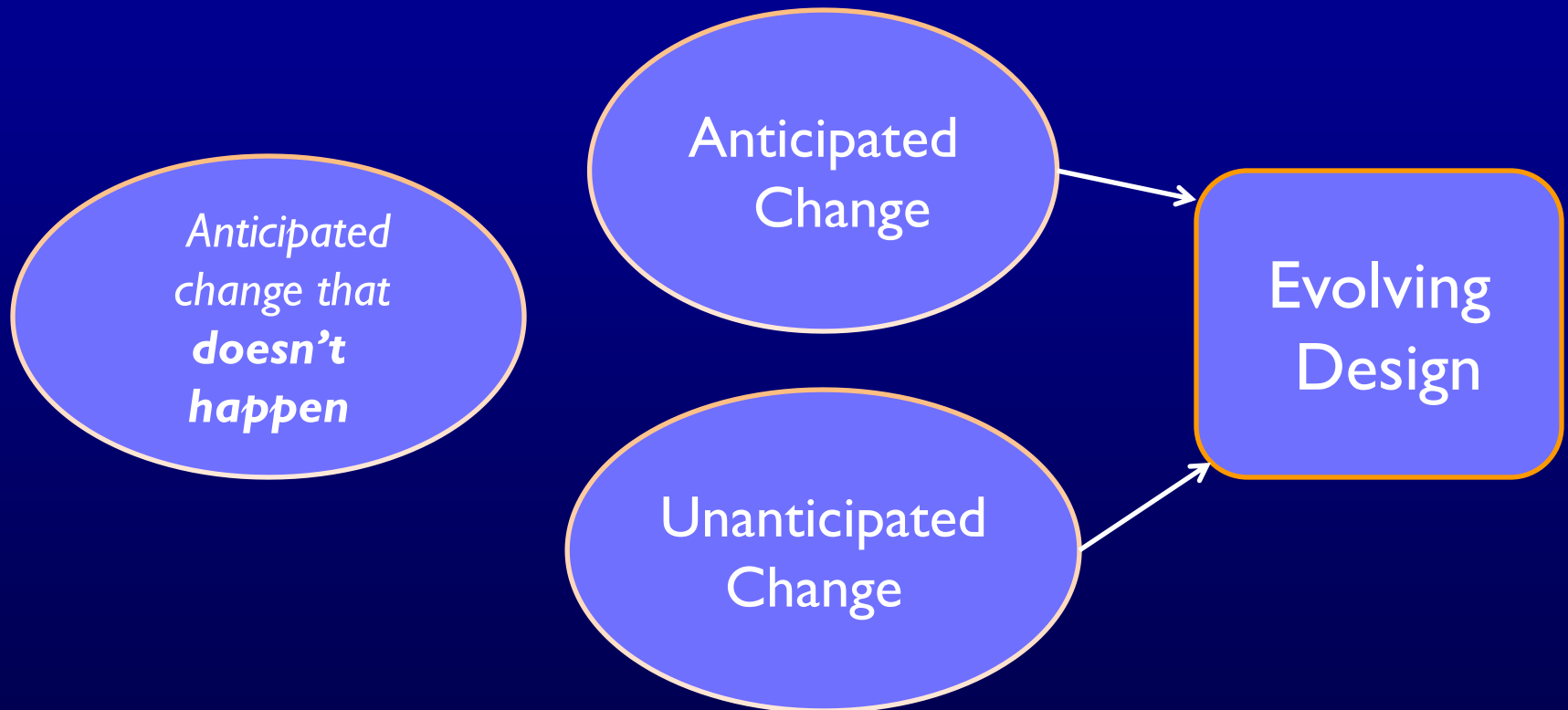
Why Be Agile ?

- Agile methods start by recognizing that **neither assumption** is valid for many current software projects
 - Software engineers are **not good at developing requirements**
 - We do not anticipate many **changes**
 - Many of the changes we do anticipate are **not needed**
- Requirements (and other “non-executable artifacts”) tend to go **out of date** very quickly
 - We seldom take time to **update** them
 - Many current software projects **change continuously**
- Agile methods expect software to **start small and evolve** over time
 - Embraces **software evolution** instead of fighting it

Supporting Evolutionary Design

Traditional design advice says to anticipate changes

Designers often anticipate changes that don't happen



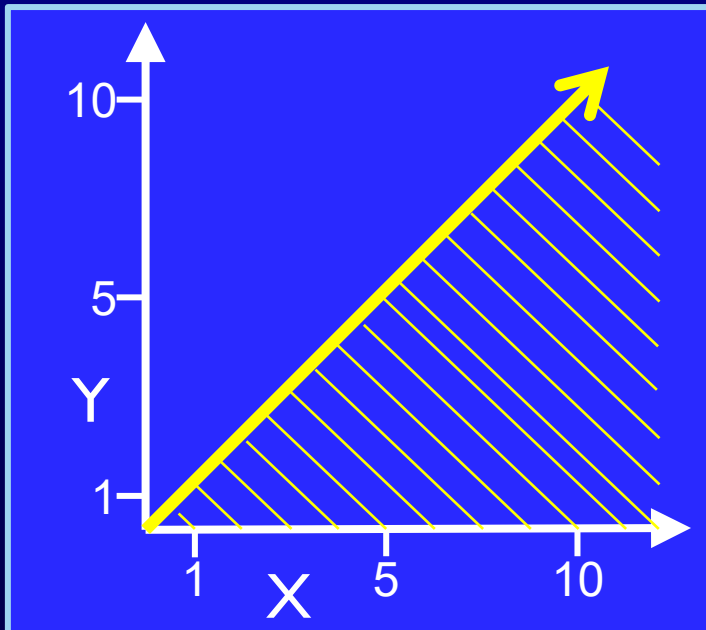
Both anticipated and unanticipated changes affect design

The Test Harness as Guardian (4.2)

What is Correctness ?

Traditional Correctness
(Universal)

$$\forall x, y, x \geq y$$



Agile Correctness
(Existential)

```
{ (1, 1) → T
  (1, 0) → T
  (0, 1) → F
  (10, 5) → T
  (10, 12) → F }
```

A Limited View of Correctness

- In **traditional** methods, we try to define **all correct behavior** completely, at the beginning
 - What is **correctness**?
 - Does “correctness” **mean anything** in large engineering products?
 - People are **VERY BAD** at completely defining correctness
- In **agile** methods, we redefine correctness to be **relative** to a specific set of tests
 - If the software behaves correctly **on the tests**, it is “correct”
 - Instead of **defining all** behaviors, we **demonstrate some** behaviors
 - **Mathematicians** may be disappointed at the lack of completeness

But software engineers ain't mathematicians!

Test Harnesses Verify Correctness

A *test harness* runs all automated tests efficiently and reports results to the developers

- Tests must be **automated**
 - Test automation is a **prerequisite** to test driven development
- Every test must include a **test oracle** that can evaluate whether that test executed correctly
- The tests replace the **requirements**
- Tests must be **high quality** and must **run quickly**
- We run tests **every time** we make a change to the software

Continuous Integration

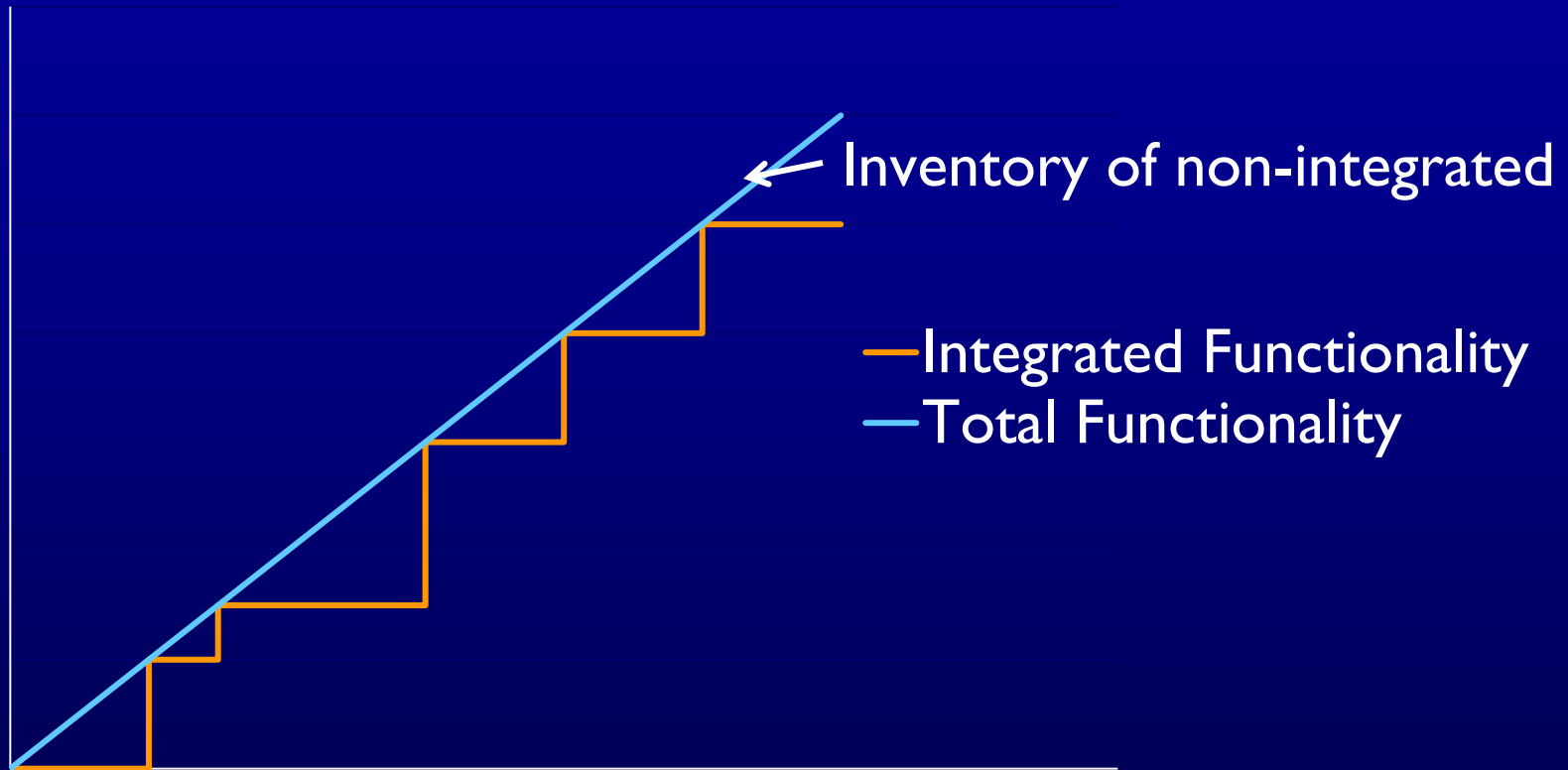
- Agile methods work best when the current version of the software can be run against all tests at any time

A *continuous integration server* rebuilds the system, returns, and re-verifies tests whenever *any* update is checked into the repository

- Mistakes are caught earlier
- Other developers are aware of changes early
- The rebuild and reverify must happen as soon as possible
 - Thus, tests need to execute quickly

A *continuous integration server* doesn't just run tests, it decides if a modified system is *still correct*

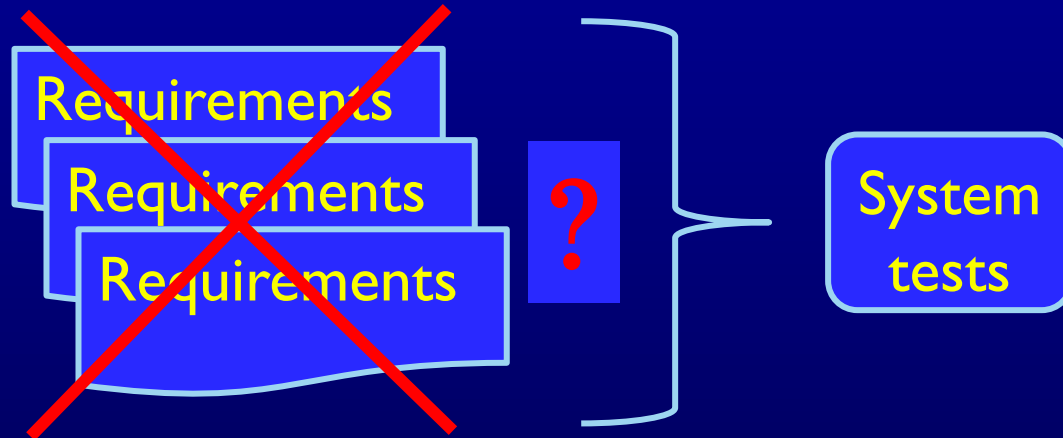
Continuous Integration Reduces Risk



Non-integrated functionality is dangerous!

System Tests in Agile Methods

Traditional testers often design system tests from requirements



But ... what if there are no traditional requirements documents ?

User Stories

A *user story* is a few sentences that captures what a user will do with the software

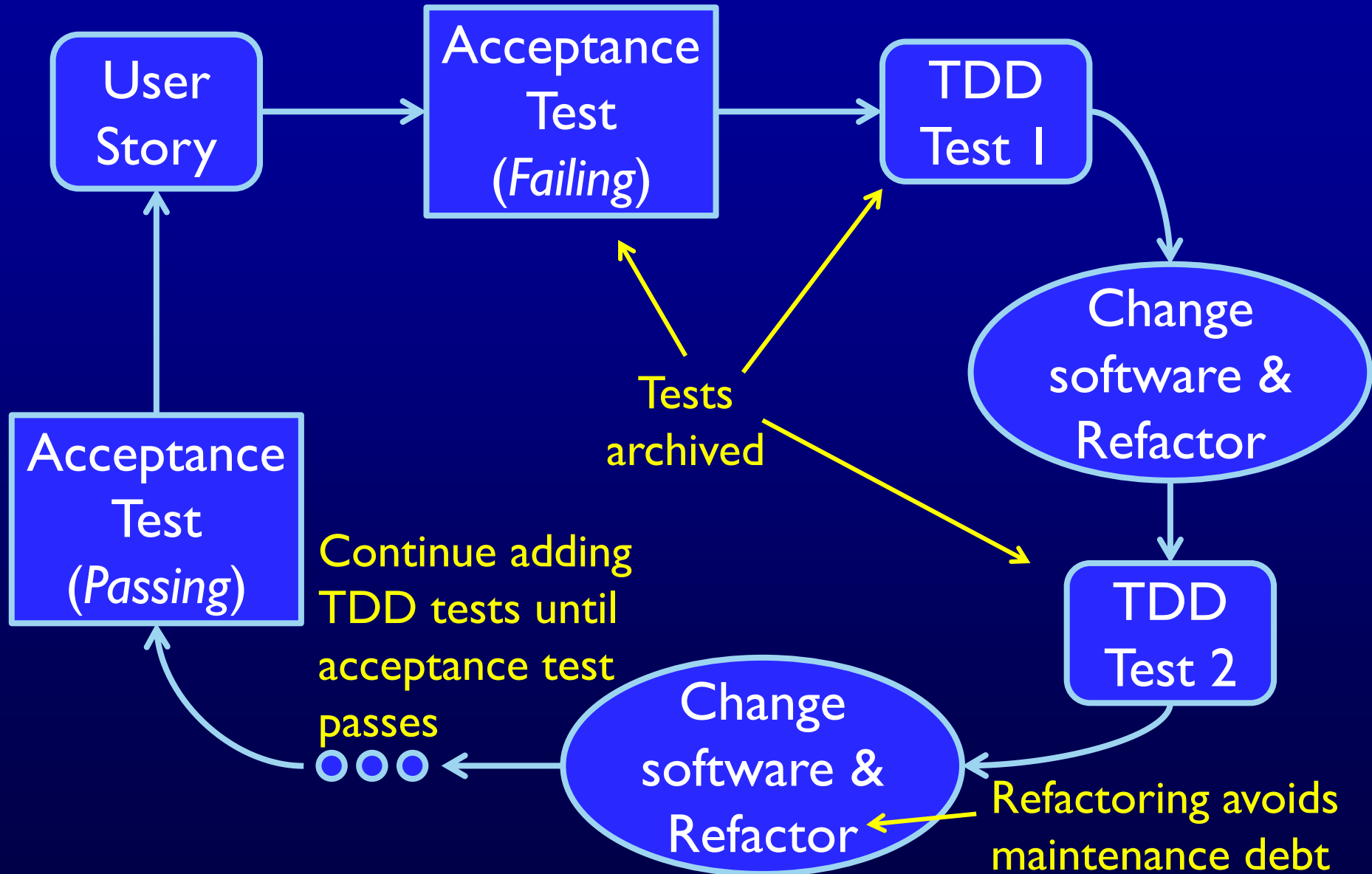
Withdraw money from
checking account

Agent sees a list of today's
interview applicants

Support technician sees
customer's history on
demand

- In the language of the **end user**
- Usually small in scale with **few details**
- **Not** archived

Acceptance Tests in Agile Methods



Adding Tests to Existing Systems

- Most of today's software is **legacy**
 - No legacy **tests**
 - Legacy requirements hopelessly **outdated**
 - Designs, if they were ever written down, **lost**
- Companies sometimes **choose not to change** software out of fear of failure

How to apply TDD to legacy software with no tests?

- Create an entire new test set? — too **expensive!**
- Give up? — a mixed project is **unmanageable**

Incremental TDD

- When a change is made, add TDD tests for **just that change**
 - Refactor
- As the project proceeds, the collection of TDD tests continues to **grow**
- Eventually the software will have **strong TDD tests**

The Testing Shortfall

- Do **TDD tests** (acceptance or otherwise) test the software well?
 - Do the tests achieve good **coverage** on the code?
 - Do the tests find most of the **faults**?
 - If the software passes, should management feel confident the software is **reliable**?

NO!



Why Not?

- Most agile tests focus on “*happy paths*”
 - What should happen under normal use
- They often miss things like
 - **Confused**-user paths
 - **Creative**-user paths
 - **Malicious**-user paths

The agile methods literature
does not give much guidance

What Should Testers Do?

Ummm ... Excuse me, Professor ...



What do I **DO**?

Design Good Tests

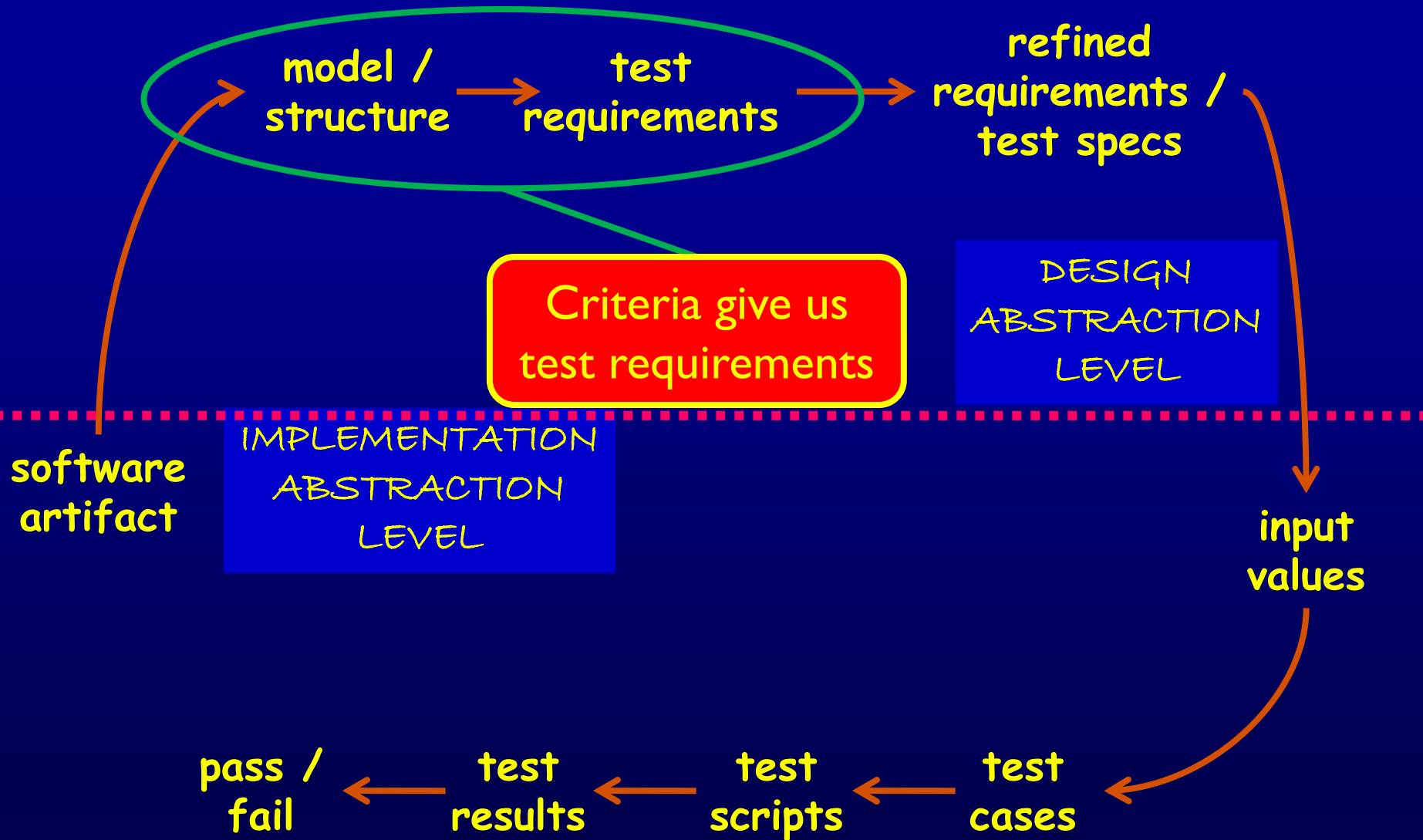
1. Use a human-based approach

- Create additional user stories that describe non-happy paths
- How do you know when you're finished?
- Some people are very good at this, some are bad, and it's hard to teach

2. Use modeling and criteria

- Model the input domain to design tests
- Model software behavior with graphs, logic, or grammars
- A built-in sense of completion
- Much easier to teach—engineering
- Requires discrete math knowledge

Model-Driven Test Design



New : Test Coverage Criteria

A tester's job is **simple** : Define a model of the software, then find ways to cover it

- **Test Requirements** : A specific element of a software artifact that a test case must satisfy or cover
- **Coverage Criterion** : A rule or collection of rules that impose test requirements on a test set

Testing researchers have defined dozens of criteria, but they are all really just a few criteria on four types of structures ...

Source of Structures

- These structures can be **extracted** from lots of software artifacts
 - **Graphs** can be extracted from UML use cases, finite state machines, source code, ...
 - **Logical expressions** can be extracted from decisions in program source, guards on transitions, conditionals in use cases, ...
- This is not the same as “***model-based testing***,” which derives tests from a model that describes some aspects of the system under test
 - The model usually describes part of the **behavior**
 - The **source** is explicitly **not** considered a model

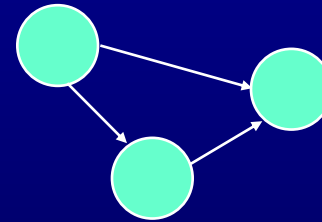
Criteria Based on Structures

Structures : Four ways to model software

1. Input Domain
Characterization
(sets)

A: {0, 1, >1}
B: {600, 700, 800}
C: {swe, cs, isa, ifs}

2. Graphs



3. Logical Expressions

(not X or not Y) and A and B

4. Syntactic Structures
(grammars)

```
if (x > y)
    z = x - y;
else
    z = 2 * x;
```


Example : Jelly Bean Coverage

Flavors :

1. Lemon
2. Pistachio
3. Cantaloupe
4. Pear
5. Tangerine
6. Apricot



Colors :

1. Yellow (Lemon, Apricot)
2. Green (Pistachio)
3. Orange (Cantaloupe, Tangerine)
4. White (Pear)

□ Possible coverage criteria :

1. Taste one jelly bean of **each flavor**
 - Deciding if yellow jelly bean is Lemon or Apricot is a controllability problem
2. Taste one jelly bean of **each color**

Coverage

Given a set of test requirements TR for coverage criterion C , a test set T satisfies C coverage if and only if for every test requirement tr in TR , there is at least one test t in T such that t satisfies tr

- **Infeasible test requirements** : test requirements that cannot be satisfied
 - No test case values exist that meet the test requirements
 - Example: Dead code
 - Detection of infeasible test requirements is formally undecidable for most test criteria
- Thus, 100% coverage is **impossible** in practice

More Jelly Beans

T1 = { three Lemons, one Pistachio, two Cantaloupes, one Pear, one Tangerine, four Apricots }

- Does test set T1 satisfy the **flavor criterion** ?

T2 = { One Lemon, two Pistachios, one Pear, three Tangerines }

- Does test set T2 satisfy the **flavor criterion** ?
- Does test set T2 satisfy the **color criterion** ?

Coverage Level

The ratio of the number of test requirements satisfied by T to the size of TR

- T2 on the previous slide satisfies 4 of 6 test requirements

Two Ways to Use Test Criteria

1. **Directly generate** test values **to satisfy** the criterion
 - Often assumed by the research community
 - Most obvious way to use criteria
 - Very hard without automated tools
2. Generate test values **externally** and **measure** against the criterion
 - Usually favored by industry
 - Sometimes misleading
 - If tests do not reach 100% coverage, what does that mean?

**Test criteria are sometimes called
metrics**

Generators and Recognizers

- **Generator** : A procedure that automatically generates values to satisfy a criterion
- **Recognizer** : A procedure that decides whether a given set of test values satisfies a criterion
- Both problems are provably **undecidable** for most criteria
- It is possible to recognize whether test cases satisfy a criterion far more often than it is possible to generate tests that satisfy the criterion
- **Coverage analysis tools** are quite plentiful

Comparing Criteria with Subsumption (5.2)

- **Criteria Subsumption** : A test criterion $C1$ subsumes $C2$ if and only if every set of test cases that satisfies criterion $C1$ also satisfies $C2$
- Must be true for **every set** of test cases
- *Examples* :
 - The flavor criterion on jelly beans subsumes the color criterion ... if we taste every flavor we taste one of every color
 - If a test set has covered every branch in a program (satisfied the branch criterion), then the test set is guaranteed to also have covered every statement

Advantages of Criteria-Based Test Design (5.3)

- Criteria maximize the “bang for the buck”
 - Fewer tests that are more effective at finding faults
- Comprehensive test set with minimal overlap
- Traceability from software artifacts to tests
 - The “why” for each test is answered
 - Built-in support for regression testing
- A “stopping rule” for testing—advance knowledge of how many tests are needed
- Natural to automate

Characteristics of a Good Coverage Criterion

1. It should be fairly easy to compute test requirements **automatically**
 2. It should be **efficient to generate** test values
 3. The resulting tests should reveal as many **faults** as possible
- Subsumption is only a **rough approximation** of fault revealing capability
 - Researchers still need to gives us more data on how to **compare** coverage criteria

Test Coverage Criteria

- ❑ Traditional software testing is **expensive** and **labor-intensive**
- ❑ Formal coverage criteria are used to decide **which test inputs** to use
- ❑ More likely that the tester will **find problems**
- ❑ Greater assurance that the software is of **high quality** and **reliability**
- ❑ A goal or **stopping rule** for testing
- ❑ Criteria makes testing more **efficient** and **effective**

How do we start applying these ideas in practice?

How to Improve Testing ?

- ❑ Testers need more and better **software tools**
- ❑ Testers need to adopt **practices and techniques** that lead to more **efficient** and **effective** testing
 - More **education**
 - Different **management** organizational strategies
- ❑ Testing & QA teams need more **technical expertise**
 - **Developer** expertise has been increasing dramatically
- ❑ Testing & QA teams need to **specialize** more
 - This same trend happened for **development** in the 1990s

Four Roadblocks to Adoption

1. Lack of test education

Microsoft and Google say half their engineers are testers, programmers test half the time

Number of UG CS programs in US that require testing ? 0

Number of MS CS programs in US that require testing ? 0

Number of UG testing classes in the US ? ~50

2. Necessity to change process

Adoption of many test techniques and tools require changes in development process

This is expensive for most software companies

3. Usability of tools

Many testing tools require the user to know the underlying theory to use them

Do we need to know how an internal combustion engine works to drive ?

Do we need to understand parsing and code generation to use a compiler ?

4. Weak and ineffective tools

Most test tools don't do much – but most users do not realize they could be better

Few tools solve the key technical problem – **generating test values automatically**

Criteria Summary

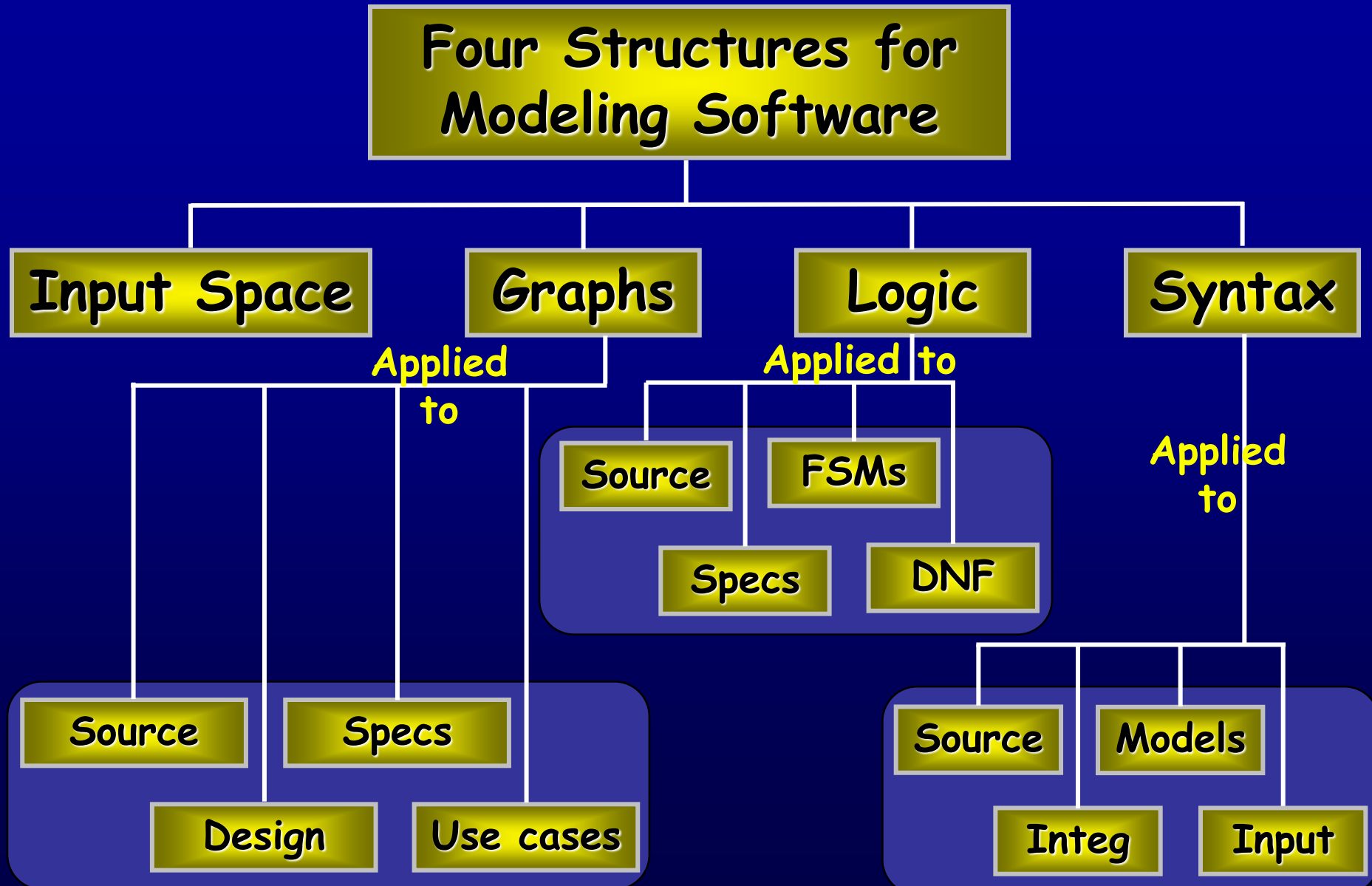
- Many companies still use “monkey testing”
 - A human sits at the keyboard, wiggles the mouse and bangs the keyboard
 - No automation
 - Minimal training required
- Some companies automate human-designed tests
- But companies that use both automation and criteria-based testing

Save money

Find more faults

Build better software

Structures for Criteria-Based Testing



Summary:

Why Do We Test Software ?

**A tester's goal is to eliminate faults
as early as possible**

- **Improve quality**
- **Reduce cost**
- **Preserve customer satisfaction**

Summary

- More companies are putting **testing first**
- This can dramatically **decrease cost** and **increase quality**
- A different view of “**correctness**”
 - Restricted but practical
- Embraces **evolutionary design**
- TDD is definitely **not** test automation
 - Test automation is a **prerequisite** to TDD
- **Agile tests** aren't enough

Summary of Lecture 1

1. **Why** do we test – to **reduce the risk** of using software
 - Faults, failures, the RIPR model
 - Test **process maturity** levels – level 4 is a **mental discipline** that improves the **quality** of the software
2. Model-Driven Test Design
 - Four types of **test activities** – test design, automation, execution and evaluation
3. Test Automation
 - Testability, observability and controllability, test automation frameworks
4. Test Driven Development
5. Criteria-based test design
 - **Four structures** – test requirements and criteria

Earlier and better testing empowers test managers